Lab Handout 1-G

Receiver Firmware: ISR-Driven Capture and Universal Asynchronous Receiver/Transmitter (UART) Forwarding

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Implement receiver firmware that captures beacon packets via an `RxDone` interrupt, queues them in a ring buffer, and forwards them from the main loop to the Field-Programmable Gate Array (FPGA) over a 43-byte framed UART protocol. This is the software side of the Week 4 deliverable.

2 Prerequisites

- Lab Handouts 1-A, 1-B, 1-C, 1-D, 1-F complete.
- At least one working beacon from LH1-F as a traffic source.

[Figure placeholder]

`LH1G_isr_mainloop_dataflow.png`

Receiver data flow: `DIO0` → `EXTI` → Interrupt Service Routine (ISR) (FIFO read, Received Signal Strength Indicator (RSSI)/Signal-to-Noise Ratio (SNR), timestamp) → ring buffer → main loop (frame build, Cyclic Redundancy Check (CRC)) → DMA UART → FPGA. Producer and consumer live in different execution contexts — the ring buffer is the sole handoff.

Figure 1: Receiver data flow: `DIO0` → `EXTI` → ISR (FIFO read, RSSI/SNR, timestamp) → ring buffer → main loop (frame build, CRC) → DMA UART → FPGA. Producer and consumer live in different execution contexts — the ring buffer is the sole handoff.

3 Forwarding Frame Format

The receiver wraps each received beacon in a framed UART message for the FPGA. Total 43 bytes:

Offset	Field	Length
0	Start delimiter (`0x7E`)	1
1	Frame length (always 41)	1
2–33	Original 32-byte beacon packet	32
34–35	RSSI (int16, LE, dBm ×1)	2
36	SNR (int8, dB ×4 per SX1276 scaling)	1
37–40	Receiver timestamp (uint32, LE, ms)	4
41–42	CRC-16/CCITT-FALSE over bytes 0–40	2

[Figure placeholder]

`LH1G_frame_layout.png`

43-byte forwarding frame layout with delimiter, length, beacon payload, RSSI/SNR, timestamp, and trailing CRC-16 highlighted.

Figure 2: 43-byte forwarding frame layout with delimiter, length, beacon payload, RSSI/SNR, timestamp, and trailing CRC-16 highlighted.

4 Architecture

4.1 ISR path

1. DIO0 rising edge triggers an `EXTI` interrupt.
2. ISR reads `RegIrqFlags`. If `RxDone` is set and `PayloadCrcError` is clear:
   (a) Read `RegFifoRxCurrentAddr`, set `RegFifoAddrPtr`, read `RegRxNbBytes` bytes from `RegFifo`.
   (b) Read `RegPktRssiValue` and `RegPktSnrValue`.
   (c) Stamp the current `ms_count`.
   (d) Push a struct (`beacon_pkt_t`) containing these fields into a *lock-free single-producer/single-consumer ring buffer* (producer = ISR, consumer = main loop).
   (e) Clear `RegIrqFlags` by writing `0xFF`.

4.2 Main-loop path

1. Check ring buffer head vs. tail. If not empty, dequeue one packet.
2. Build the 43-byte forwarding frame.
3. Compute CRC-16/CCITT-FALSE over bytes 0–40 and place in bytes 41–42.
4. Transmit the entire 43-byte frame over `USART3` (or whichever UART is wired to the Nexys A7) using `HAL_UART_Transmit_DMA`.
5. Increment a `frames_forwarded` counter.

[Figure placeholder]

`LH1G_ring_buffer_diagram.png`

Single-producer/single-consumer ring buffer: ISR advances `head`, main loop advances `tail`. A `__DMB()` barrier orders the payload write against the head increment.

Figure 3: Single-producer/single-consumer ring buffer: ISR advances `head`, main loop advances `tail`. A `__DMB()` barrier orders the payload write against the head increment.

4.3 Ring buffer sizing

A 16-entry ring (≈640 bytes of beacon structs) accommodates roughly 16 back-to-back receptions without overflow. With two beacons at 2 and 3 second intervals, this is ∼40 s of margin against a stalled main loop — comfortable.

5 Procedure

5.1 Step 1 — Peripheral configuration

1. Starting from your LH1-F project or a new one, configure:
   - `SPI1` (unchanged from LH1-C).
   - `EXTI` on `PA8` (DIO0), rising-edge trigger.
   - `USART3` at 115200 8N1 on `PC4`/`PC5` (TX/RX), with TX DMA enabled.
   - `USART2` still enabled for `printf` debug over the ST-LINK virtual COM port.
   - `TIM2` 1 ms tick (unchanged).

5.2 Step 2 — Ring buffer implementation

1. Define the packet struct:

```
typedef struct {
    uint8_t  raw[32];
    int16_t  rssi_dbm;
    int8_t   snr_q4;
    uint32_t rx_timestamp_ms;
} beacon_pkt_t;
```

2. Static ring with power-of-two size: `static beacon_pkt_t ring[16];` and `volatile uint32_t head, tail;`.
3. `push`: producer (ISR). Fails silently on overflow and increments a `dropped` counter.
4. `pop`: consumer (main loop). Returns bool indicating whether a packet was dequeued.
5. Use a single `__DMB()` memory barrier between buffer write and head increment to ensure ordering on Cortex-M4.

5.3 Step 3 — ISR

1. Implement `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`. Check that the pin matches DIO0.
2. Keep the ISR short: FIFO read, RSSI/SNR read, timestamp, `push` to ring, clear IRQ flags. No `printf` inside the ISR.

5.4 Step 4 — Frame builder and forwarder

1. In the main loop, call `pop`. If a packet is present, construct the 43-byte frame as described above.
2. Reuse the CRC-16 helper from LH1-F.
3. Call `HAL_UART_Transmit_DMA(&huart3, frame, 43)`. Track the DMA completion flag; do not overwrite the frame buffer until transmission is complete.
4. Also `printf` a human-readable summary over USART2 for debug: `"RX beacon=%02X seq=%lu rssi=%d snr=%d"`.

[Figure placeholder]

`LH1G_python_parser_output.png`

Host-side `pyserial` parser output: 43-byte frames captured, start delimiter matched, CRC-16 validated, beacon ID/sequence/RSSI/SNR extracted.

Figure 4: Host-side `pyserial` parser output: 43-byte frames captured, start delimiter matched, CRC-16 validated, beacon ID/sequence/RSSI/SNR extracted.

5.5 Step 5 — Validation without the FPGA

1. Before wiring to the Nexys A7, loopback-test the forwarding UART by connecting `USART3 TX` to `USART3 RX` (or to a USB-UART adapter) and capturing frames on a host PC.
2. Use a Python script (`pyserial`) to parse incoming 43-byte frames, verify the start delimiter, validate the CRC-16, and print the extracted beacon ID, sequence number, RSSI, and SNR.
3. Run with Beacon A (and optionally Beacon B) from LH1-F broadcasting.

6 Verification Checklist

- ☐ `RxDone` interrupt fires on every received packet (ISR entry count matches expected receptions).
- ☐ Ring buffer overflows only if you deliberately stall the main loop.
- ☐ 43-byte frames appear on `USART3 TX` for every received beacon.
- ☐ Host Python parser validates start delimiter and CRC-16 on 100% of frames.
- ☐ Dual-beacon run: parser sees interleaved ID = `0x01` and ID = `0x02` frames with independently contiguous sequence numbers.
- ☐ Dropped-packet counter remains zero during a 5-minute run.

Concurrency Hazards

- Access `head`/`tail` as `volatile`. Never read the same index twice in the consumer without rechecking.
- Do not call Hardware Abstraction Layer (HAL) `printf` from inside the ISR — it blocks on UART transmit and will drop RF packets.
- Do not overwrite the DMA source buffer while a prior transmission is still in flight. Use a simple busy flag or a second buffer.

Lab Handout 1-G Deliverable

1. Full receiver firmware source, including ring buffer and ISR.
2. Python parser script (one file) that validates frames from `USART3`.
3. 5-minute dual-beacon log showing zero dropped packets and zero CRC failures in the parser.
4. One-paragraph rationale for the ring buffer size you chose, including a worst-case arrival-rate analysis.

Glossary

CRC	Cyclic Redundancy Check	1, 2, 3, 4, 5, 6
FPGA	Field-Programmable Gate Array	1, 2, 5
HAL	Hardware Abstraction Layer	6
ISR	Interrupt Service Routine	1, 3, 4, 6
RF	Radio Frequency	1, 6
RSSI	Received Signal Strength Indicator	1, 2, 4, 5
SNR	Signal-to-Noise Ratio	1, 2, 4, 5
UART	Universal Asynchronous Receiver/Transmitter	1, 2, 3, 5